

TCP Sockets Project

main_server.c:

```
void error(const char *msg) { perror(msg); exit(1); }
```

- This prints an error message using the perror() function as this shows system level error information
- This also appends a text from errno to show the user that there is an error
- This also terminates the program using exit(1)
- This function is intended for fatal errors that prevent the server from proceeding

```
Client* find_client_in_room(Room* r, const char* username) {
```

- This function iterates through the linked list of clients in the room
- And it also compares each client's username with the target name
- As a result, it returns the matching "Client*" if found
- As a result, it also returns "NULL" if no such room exists

```
int create_room_locked() {
```

- This function assumes the mutex is already locked before being called
- This function creates a new room if there's space remaining
- It also assigns a unique ID where it starts with 1 or "last_room_id" + 1.
- It increments global "room_count" and returns the new room ID.

```
void add_client_to_room_locked(Room* r, Client* c) {
```

- This function sets the client's "room_id" to the room's ID
- And it assigns a unique color index by calling "assign_color_idx_locked()
- And it ensures that new clients are fully integrated into the room
- And inserts the client at the head of the room's linked list.

```
void* client_thread(void* v) {
```

- This function is used for messaging, file sending, and for quitting the chat program
- This function performs initial handshake such as: NEW/JOIN/LIST under "rooms_lock" by creating or joining rooms
- Then it receives and sets the client's name, and it assigns a color, and it announces join with "broadcast_to_room"
- Then it enters a loop handling such as: MSG,SEND, and QUIT commands and generates file transfer threads as needed.

- When exiting, it removes the client from the room and broadcasts the leave notice

```
void* file_transfer_thread(void* arg) {
```

- This function runs in its own thread accepts a “FileTransferArg* argument
- And it attempts to “fopen()” the given file and sends the error to send and exits if it fails
- It also notifies the receiver with and accept or deny prompt and “recv()” their response
- And if accepted, it reads the file in “FILE_BUF” and sends them to the receiver

```
// Debug print of all rooms + clients
void server_print_client_list_locked() {
```

- This function prints a header and iterates all rooms from rooms[0] to rooms[room_count -1]
- It iterates that room’s client and prints the username and “ipstr”
- And then it calls “fflush(stdout) to ensure output appears correctly
- And it is designed for operator debugging and the state visibility

```
int main() {
```

- This main function creates a TCP listening socket and it binds to “PORT_NUM”
- Then it calls the listen() function to start accepting connections and prints a start up message
- And it ensures at least one chat room exists by calling “create_room_locked()” under the mutex
- It accepts incoming connections in a loop and prints the client IP and creates a client thread for each

main_client.c:

```
void error(const char* msg){ perror(msg); exit(0); }
```

- This function prints a human readable message that includes system errno using perror() function
- And it signals fatal problem and immediately terminates the process using exit(0)
- It also provides a single consistent place to fail fast on connection errors
- This is best used for unrecoverable client errors such as invalid args or socket failures.

```
void* recv_thread(void* arg){
```

- This runs in a dedicated thread to receive and print the data from the server using recv() function
- Then it frees the ThreadArgs* passed in immediately
- It also detects the file transfer prompts by searching for Y/N and it prints the prompt and reads the user’s Y/N response from stdin and sends back to the server
- This function also prints any other incoming messages to stdout and flushes so output appears immediately

```
void* send_thread(void* arg){
```

- This runs in a dedicated thread to read user input from stdin and send commands and messages to the server
- This function treated an empty line as a quit signal and sends “QUIT\n” to the server before exiting
- For lines beginning with SEND, it forwards the raw commands to the server for other lines wrap text as “MSG <text>\n”
- This function also strips the trailing newline from user input before processing so protocol messages are all well formed

```
int main(int argc, char* argv[]){
```

- This main function validates the command line arguments and opens a TCP socket to argv[1] on “PORT_NUM”
- And then it performs room selection, if a room argument is provided it uses NEW or JOIN<room>, requests LIST from server and displays option and reads the user’s choice
- Then it starts two threads, one for receiving and one for sending. Passing each a ThreadArgs containing the socket
- Then it joins both threads, closes the socket and then returns when both threads finish.

Outputs:

OUTPUT	DEBUG CONSOLE	PORTS	TERMINAL
<pre>(base) abishekk_k@Abishekks-MacBook-Pro-2 PR PROJECT405 % gcc main_server.c -lpthread -o ma in_server (base) abishekk_k@Abishekks-MacBook-Pro-2 PR PROJECT405 % ./main_server main_server: listening on port 1004 Connected: 127.0.0.1 === Server: current rooms and participants = === Room 1: 1 people - Alice (127.0.0.1) Connected: 127.0.0.1 === Server: current rooms and participants = === Room 1: 2 people - Bob (127.0.0.1) - Alice (127.0.0.1) []</pre>	<pre>(base) abishekk_k@Abishekks-MacBook-Pro-2 PR PROJECT405 % gcc main_client.c -lpthread -o ma in_client (base) abishekk_k@Abishekks-MacBook-Pro-2 PR PROJECT405 % ./main_client 127.0.0.1 Server says following options are available: Room 1: 0 people Choose the room number or type [new] to crea te a new room: 1 ROOM 1 Type your user name: Alice Alice (127.0.0.1) joined the chat room! Bob (127.0.0.1) joined the chat room! [Bob] hello everyone hi bob how are you [Bob] im good []</pre>	<pre>(base) abishekk_k@Abishekks-MacBook-Pro-2 PR PROJECT405 % ./main_client 127.0.0.1 Server says following options are availabl e: Room 1: 1 person Choose the room number or type [new] to cr eate a new room: 1 ROOM 1 Type your user name: Bob Bob (127.0.0.1) joined the chat room! hello everyone [Alice] hi bob [Alice] how are you im good []</pre>	

- This output shows that the server starts on port 1004 and waits for incoming connections
- Then when a client connects, the server prints the current room list
- Then the client is prompted to choose a room or create a new one if no room exist, then they type in their username
- After joining, the server updates the room and displays the current list of participants.
- Clients also see message from one another inside the same room

```
OUTPUT  DEBUG CONSOLE  PORTS  TERMINAL

Room 3: 1 people
- Bob (127.0.0.1)
Room 4: 1 people
- (no-name) (127.0.0.1)
=== Server: current rooms and participants ===
Room 1: 0 people
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
- (no-name) (127.0.0.1)
=== Server: current rooms and participants ===
Room 1: 0 people
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 3 people
- Tim (127.0.0.1)
- Bob (127.0.0.1)
- Alice (127.0.0.1)
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
[]

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 0 people
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Choose the room number or type [new] to create a new room:
1
ROOM 1
Type your user name: Alice
Alice (127.0.0.1) joined the chat room!
Bob (127.0.0.1) joined the chat room!
Tim (127.0.0.1) joined the chat room!
[]

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Choose the room number or type [new] to create a new room:
1
ROOM 1
Type your user name: Bob
Bob (127.0.0.1) joined the chat room!
Tim (127.0.0.1) joined the chat room!
[]

o (base) abishekk_k@Abishekk-Mac
cBook-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 2 people
Room 2: 0 people
Room 3: 0 people
Room 4: 0 people
Choose the room number or type [new] to create a new room:
1
ROOM 1
Type your user name: Tim
Tim (127.0.0.1) joined the chat room!
[]

Ln 313, Col 1  Spaces: 4  UTF-8  LF
```

- This output shows that multiple clients run in separate terminals and connects to the same server
- The server updates the room list dynamically as new clients join as this shows users in each room
- Then the clients joining different rooms don't see each other's chat confirming room isolation
- And when the client joins, the server prints an updates room/ participant table
- Supports independent room management

```
(base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ls -l
-rwxr-xr-x 1 abishekk_k 1004

COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE
main_serv 80677 abishekk_k 3  u  IPv4 0xc680fa2a558276eb 0
0  TCP *:*1004 (LISTEN)

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % kill -9 80677

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ./main_server
main_server: listening on port 1004
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 0 people
Room 2: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Room 2: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
Room 2: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
Room 2: 1 people
- Tim (127.0.0.1)
[]

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 0 people
Room 2: 0 people
Choose the room number or type [new] to create a new room:
1
ROOM 1
Type your user name: Alice
Alice (127.0.0.1) joined the chat room!
Bob (127.0.0.1) joined the chat room!
[]

o (base) abishekk_k@Abishekk-Mac
Book-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Room 2: 0 people
Choose the room number or type [new] to create a new room:
1
ROOM 1
Type your user name: Bob
Bob (127.0.0.1) joined the chat room!
[]

o (base) abishekk_k@Abishekk-Mac
cBook-Pro-2 PROJECT405 % ./main_client 127.0.0.1
Try connecting to 127.0.0.1...
Server says following options are available:
Room 1: 2 people
Room 2: 0 people
Choose the room number or type [new] to create a new room:
2
ROOM 2
Type your user name: Tim
Tim (127.0.0.1) joined the chat room!
[]

zsh PRO...
zsh PRO...
zsh PRO...
zsh PRO...

Ln 313, Col 1  Spaces: 4  UTF-8  LF  {} C  Mac
```

- This output shows when the client runs without specifying a room or using “new”, the server sends a list of all available rooms
- Then the client must choose a room number or type in “new” for UI completeness
- If the user chooses an existing room, the server puts them into that room and notifies all users
- If the room is empty or doesn’t exist, then the server handles creation or error messages as needed
- There are also multiple side to side clients displaying accurate room roster and join notifications

```

OUTPUT  DEBUG CONSOLE  PORTS  TERMINAL

Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
echo "Hello Bob, this is a test file!" > testfile.txt
ls
=== Server: current rooms and participants ===
Room 1: 1 people
- Bob (127.0.0.1)
=== Server: current rooms and participants ===
Room 1: 0 people
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
=== Server: current rooms and participants ===
Room 1: 1 people
- Alice (127.0.0.1)
Connected: 127.0.0.1
=== Server: current rooms and participants ===
Room 1: 2 people
- Alice (127.0.0.1)
- Bob (127.0.0.1)

chat_client.c      chat_server_full.c  mai
n_server
chat_client_full.c  main_client        mai
n_server.c
(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT40
S % ./main_client 127.0.0.1

Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 0 people
Choose the room number or type [new] to create a new room:
1
Type your user name: Alice
ROOM 1
Alice (127.0.0.1) joined the chat room!
Bob (127.0.0.1) joined the chat room!
Bob (127.0.0.1) left the room!
Bob (127.0.0.1) joined the chat room!

(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT40
S % ls
Makefile      main_client
chat_client.c  main_client.c
chat_client_full.c  main_server
chat_server.c  main_server.c
chat_server_full.c  testfile.txt
(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT40
S % SEND Bob testfile.txt

zsh: command not found: SEND
(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT40
S % ./main_client 127.0.0.1
Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room:
1
Type your user name: Alice
ROOM 1
Alice (127.0.0.1) joined the chat room!
SEND Bob testfile.txt
[Bob] Y

(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT4
05 % ./main_client 127.0.0.1

Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room:
1
Type your user name: Bob
ROOM 1
Bob (127.0.0.1) joined the chat room!

(base) abishekk_k@Abishekks-MacBook-Pro-2 PROJECT4
05 % ./main_client 127.0.0.1

Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room:
1
Type your user name: Bob
ROOM 1
Bob (127.0.0.1) joined the chat room!
Alice (127.0.0.1) left the room!
Alice (127.0.0.1) joined the chat room!
Alice wants to send you file testfile.txt. Accept? (Y/N)
Y
Hello Bob, this is a test file!
File transfer complete.

```

- This output shows that the client supports a special message format where you can send and receive files.
- The server notifies the receiving user and asks to either accept or reject it
- And if accepted, the file is streamed in binary safe mode and saved in the receiver’s directory
- The transfer finishes with “file transfer complete” and the receiver also sees the full file contents.

AI USE:

Prompt I used:

help me understand checkpoint4:(15 points) Allow file transfer. A client can send a local file to a remote client with a special command [SEND]. The syntax is like below
[in client terminal]
[this-client-IP-or-user-name]: SEND receiving-user-name file-name
The receiving client should be notified of the transfer and agree on receiving the file, by
Y/N question. When the receiving client types, "Y", then the transmission starts. You can
assume that the file being sent is always in the same directory as the sender chat client
program. There is no assumption of file type. A binary file should be properly transmitted,
if requested. Keep in mind that the details are in the details. For example:
o If you test two clients running on the same machine, running from the same directory,
you may accidentally overwrite the source file. This should not happen. It is best to
test by running two clients running from different directories so that you can confirm
that the file from one directory has been copied to the other.
o Clients connected to the same server room should not know about the file transfer if
they are not the sender/receiver of the file. Also, a user should not be able to send a
message or a file to a user who is not in the same chat room. This is a serious security
vulnerability that was often found in early commercial chat applications.
o Be aware of the potential deadlock. There is also a potential race condition depending
on your implementation. For example, if sender or receiver drops the connection
unexpectedly in the middle of transmission, the transmission should be terminated
gracefully on the other end.
o It is possible that non-file-transfer-involved user may send a message to the server
while the file transmission is ongoing! Their communication should not be blocked
by the file transfer.
o It is possible multiple clients may try to initiate independent transfers. Hence, proper
and accurate implementation of this is to create a separate asynchronous thread that
will handle file transfers, and notify the chatting thread when the

Answer I got:

 **Summary of What You Must Build**

✓ **Sender side**

- Parses "SEND" command.
- Checks file exists locally.
- Opens file, reads bytes.
- Sends data in chunks in a separate thread.

✓ **Server side**

- Validates room membership.
- Forwards "send offer" to receiver.
- Forwards "Y/N" response.
- Forwards file bytes.
- Does not block other room users.
- Supports multiple transfers concurrently.

✓ **Receiver side**

- Prompts user for Y/N.
- If yes, opens new file.
- Streams data into file.
- Saves successfully.

 **THE BIG REQUIREMENT: ASYNCHRONOUS THREADING**

The assignment insists:

! **The file transfer must not block chat messages.**

This means:

- Sending thread = separate thread
- Receiving thread = separate thread
- Chat thread stays active

If Bob is sending a file to Alice:

- Charlie (same room) can still send chat messages.
- David (another room) should not even know file transfer exists.

This is why they warn about race conditions and deadlocks.

 **Important Pitfalls You MUST avoid**

1. Don't block the whole client while sending a file

If you send bytes in the same thread as your chat, everything freezes.

You must use a **separate pthread** like:

```
c
pthread_create(&file_send_thread, NULL, send_file_func, args);
```

[Copy code](#)

Why I used AI for this Checkpoint:

I used AI to understand the detailed description of checkpoint 4, as I kept getting errors when I was running my code. Every time I typed in SEND a file or when I was trying to receive a txt file from a client, nothing would show up on my terminal. And in order to fix that AI told me that I need to write functions that will support file transfer. So I wrote the file transfer code based on the skeleton code/pseudocode it gave me.

can you give me a pseduocode for the file transfer part in c

Absolutely — here is a clean, correct, assignment-ready pseudocode for Checkpoint 4's file-transfer system.

It covers client sender, client receiver, and server, and includes threading, room checking, and safe file writing.

✅ FILE TRANSFER PSEUDOCODE (Checkpoint 4 — C)

CLIENT SIDE: MAIN LOOP

```
pseudo

loop forever:
    user_input = read_from_stdin()

    if user_input starts with "SEND":
        parse recipient and filename
        start thread: send_file_request_thread(recipient, filename)
    else:
        send normal chat message to server
```

CLIENT: SEND FILE REQUEST THREAD

```
in
thread send_file_request_thread(recipient, filename):

    if file does not exist locally:
        print("ERROR: file not found")
        return

    // send request to server
    send_to_server("SEND_REQUEST new-<recipient> <filename>")

    // wait for server reply: ACCEPT or DENY
    msg = recv_from_server()

    if msg == "SEND_ACCEPT":
        print("Recipient rejected file.")
        return

    if msg == "SEND_ACCEPTED":
        start thread: send_file_data_thread(filename)
```

CLIENT: SEND FILE DATA THREAD

```
in
thread send_file_data_thread(filename):

    open file for reading in binary mode

    send_to_server("FILE_BEGIN <filename> <filesize>")

    while not end_of_file:
        chunk = read next 1024 bytes
        send_to_server("FILE_DATA <binary chunk>")

    send_to_server("FILE_END")

    close file

    print("File transfer completed.")
```

CLIENT: RECEIVE SIDE (inside network read thread)

Your client already has a thread constantly reading messages from server.

```
Extend it:

pseudo

thread receive_from_server_thread:

    loop forever:
        msg = recv_from_server()

        if msg starts with "SEND_OFFER":
            parse sender and filename
            print "sender wants to send you filename. Accept? (Y/N)"

            wait for user_input
            if input == "Y":
                send_to_server("SEND_ACCEPT <senders>")
                start thread: receive_file_thread(filename)
            else:
                send_to_server("SEND_REJECT <senders>")

        else if msg == "FILE_BEGIN":
            parse filename, filesize
            open new file: "received_<filename>" in binary write mode
            receiving = TRUE

        else if msg == "FILE_DATA":
            write binary chunk to opened file

        else if msg == "FILE_END":
            close file
            print("File received successfully.")
            receiving = FALSE

        else:
            handle_normal_chat_message(msg)
```

RECEIVER: FILE RECEIVE THREAD

(If you want cleaner structure, separate thread)

```
pseudo

thread receive_file_thread(filename):

    open new file "received_<filename>" for writing

    loop:
        msg = recv_from_server()

        if msg starts with "FILE_DATA":
            extract binary chunk
            write to file

        if msg == "FILE_END":
            break

    close file
    print("File saved as received_<filename>")
```


Using these pseudo code examples, I was able to write this following code:

```
void* file_transfer_thread(void* arg) {
    FileTransferArg* f = (FileTransferArg*)arg;
    char buf[FILE_BUF];

    FILE* fp = fopen(f->filename, "rb");
    if (!fp) {
        char msg[128];
        snprintf(msg, sizeof(msg), "ERR cannot open file\n");
        send(f->sender->sockfd, msg, strlen(msg), 0);
        free(f);
        return NULL;
    }

    // Send notification
    char notif[BUF_SIZE];
    snprintf(notif, sizeof(notif),
        "%s wants to send you file %s. Accept? (Y/N)\n",
        f->sender->username, f->filename);
    send(f->receiver->sockfd, notif, strlen(notif), 0);

    // Wait for response
    int n = recv(f->receiver->sockfd, buf, sizeof(buf) - 1, 0);
    if (n <= 0) { fclose(fp); free(f); return NULL; }
    buf[n] = '\0';
    if (buf[n-1] == '\n') buf[n-1] = '\0';

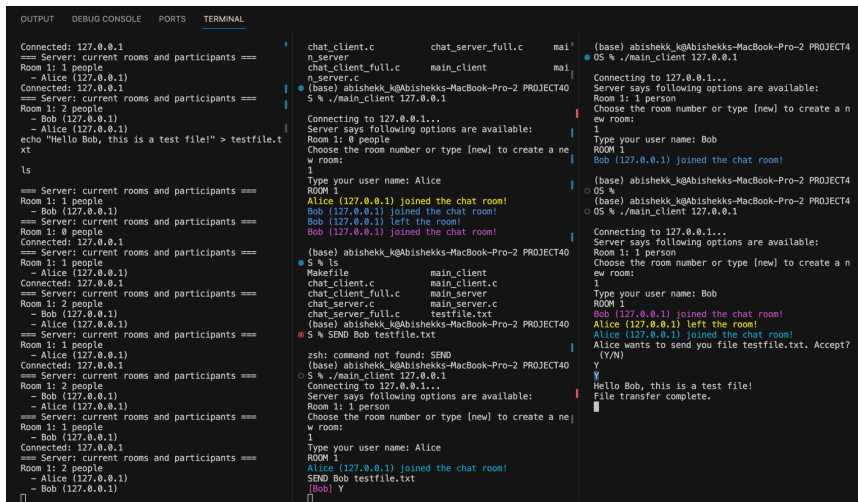
    // Abort if it is denied
    if (buf[0] != 'Y' && buf[0] != 'y') {
        fclose(fp);
        free(f);
        return NULL;
    }

    // Transfer file
    while (1) {
        size_t r = fread(buf, 1, sizeof(buf), fp);
        if (r > 0)
            send(f->receiver->sockfd, buf, r, 0);
        if (r < sizeof(buf)) break;
    }

    fclose(fp);
    char done[] = "File transfer complete.\n";
    send(f->receiver->sockfd, done, strlen(done), 0);

    free(f);
    return NULL;
}
```

Overall, AI helped me understand how to tackle such problems by using pseudo code first and gave me tips and tricks to write a working file transfer function for this project. As a result I was able to get this output:



```
OUTPUT  DEBUG CONSOLE  PORTS  TERMINAL

Connected: 127.0.0.1
== Server: current rooms and participants ==
Room 1: 1 person
- Alice (127.0.0.1)
Connected: 127.0.0.1
== Server: current rooms and participants ==
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
echo "Hello Bob, this is a test file!" > testfile.txt
ls
== Server: current rooms and participants ==
Room 1: 1 person
- Bob (127.0.0.1)
== Server: current rooms and participants ==
Room 1: 0 people
Connected: 127.0.0.1
== Server: current rooms and participants ==
Room 1: 1 person
- Alice (127.0.0.1)
Connected: 127.0.0.1
== Server: current rooms and participants ==
Room 1: 2 people
- Bob (127.0.0.1)
- Alice (127.0.0.1)
== Server: current rooms and participants ==
Room 1: 1 person
- Alice (127.0.0.1)
Connected: 127.0.0.1
== Server: current rooms and participants ==
Room 1: 2 people
- Alice (127.0.0.1)
- Bob (127.0.0.1)

chat_client.c  chat_server_full.c  mai
n_server
chat_client_full.c  main_client  mai
n_server.c
(base) abishekk_k@Abishekk-MacBook-Pro-2 PROJECT40
$ % ./main_client 127.0.0.1

Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 0 people
Choose the room number or type [new] to create a new room:
1
Type your user name: Alice
Alice (127.0.0.1) joined the chat room!
Bob (127.0.0.1) joined the chat room!
Bob (127.0.0.1) left the room!
(base) abishekk_k@Abishekk-MacBook-Pro-2 PROJECT40
$ % ls
Makefile  main_client
chat_client.c  main_server.c
chat_server.c  main_server.c
chat_server_full.c  testfile.txt
(base) abishekk_k@Abishekk-MacBook-Pro-2 PROJECT40
$ % SEND Bob testfile.txt

zsh: command not found: SEND
(base) abishekk_k@Abishekk-MacBook-Pro-2 PROJECT40
$ % ./main_client 127.0.0.1
Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room:
1
Type your user name: Alice
ROOM 1
Alice (127.0.0.1) joined the chat room!
SEND Bob testfile.txt
[Bob] Y
Y
(base) abishekk_k@Abishekk-MacBook-Pro-2 PROJECT4
$ % ./main_client 127.0.0.1
Connecting to 127.0.0.1...
Server says following options are available:
Room 1: 1 person
Choose the room number or type [new] to create a new room:
1
Type your user name: Bob
ROOM 1
Bob (127.0.0.1) joined the chat room!
Alice (127.0.0.1) left the room!
Alice (127.0.0.1) joined the chat room!
Alice wants to send you file testfile.txt. Accept?
(Y/N)
Y
Hello Bob, this is a test file!
File transfer complete.
```